INTERNAL CAPABILITY REVIEW

SKILL-BASED QA PLANNING

How QA planning shifts from analyst-led synthesis to a structured, skill-driven operating model.

01
Before LLM
Manual and analyst-dependent

02
After LLM
Prompt + skill + constraints

03
Orchestrator
Phased, gated workflow

04
Why It Works
Discipline creates quality

05
Evaluation
Champion vs challenger loop

06
Limitations
Strong, but still scoped

Case example: `qa-plan-orchestrator`

6-slide story

BEFORE LLM: MANUAL QA PLANNING

QA planning depended on manual synthesis and personal judgment.

INPUTS

Scattered sources

Jira · PRD · PR diff · Slack · Past bugs

→

STEP 1

Manual reading

Analyst reviews every artifact separately and rebuilds context by hand.

→

STEP 2

Coverage mapping

Scenarios and risk areas are inferred case by case, without a stable scaffold.

→

OUTPUT

Hand-built plan

Quality varies by analyst experience, time available, and memory of prior misses.

Slow

Every request starts from raw evidence review.

Variable

Coverage depends on who writes the plan.

Hard to reuse

Context and reasoning are rarely preserved cleanly.

AFTER LLM: PROMPT + SKILL + CONSTRAINTS

Good prompting turns planning into a structured operating model.

PROMPT INPUTS

Skill
Specify which capability to invoke.

Requirements
Lock mandatory rules and output format.

Materials
Attach tickets, diffs, docs, or examples.

Context
Provide feature family and domain grounding.

qa-plan-orchestrator

OUTPUT

Structured QA Plan
Evidence-backed scenarios, explicit grouping, review gates, and consistent plan shape.

Consistent
Same workflow every run

Traceable
Artifacts and context retained

Reusable
Evidence can be refreshed

Reviewable
Gates enforce quality

Use the skill
Name the target capability explicitly.

Set constraints
State mandatory requirements and exclusions.

Attach evidence
Provide the actual materials, not summaries only.

Define output
Ask for the plan format and level of detail you need.

HOW QA-PLAN-ORCHESTRATOR WORKS

This is a phased system with explicit artifacts and review gates.

PHASE 0
Setup
Initialize runtime and classify state.

→

PHASE 1
Evidence
Spawn requests by source family.

→

PHASE 2
Artifact Index
Build traceable lookup of collected context.

→

PHASE 3
Research
Map coverage with deep research inputs.

→

PHASE 4
Drafting
Generate subcategory and grouped plan drafts.

→

PHASE 5
Checkpoints
Full-context and shipment review passes.

→

PHASE 6
Quality Pass
Final layering, search, and few-shot cleanup.

→

PHASE 7
Finalize
Promote final plan and generate summary output.

phase1_spawn_manifest.json
Defines the evidence-collection subagent requests.

coverage_ledger
Tracks how research and evidence map to scenario coverage.

qa_plan_final.md
Promoted final artifact after review and quality gates pass.

Script-driven
Phase logic lives in scripts, not ad hoc prompting.

Spawn-based
Subtasks are launched from phase manifests.

Gate-enforced
Later phases only proceed when validation passes.

Artifact-first
Evidence, deltas, drafts, and final output are all persisted.

WHY THIS SKILL

Planning quality comes from workflow discipline, not model cleverness alone.

01

Script-driven

The orchestrator calls explicit phase scripts instead of improvising the workflow inline.

02

Phase gates

Each phase has a contract, so weak evidence or broken drafts get blocked before promotion.

03

Reusable evidence

Research, lookup, and support context persist across runs instead of being rebuilt every time.

04

Review checkpoints

Full-context review and shipment checkpoints force quality improvement before finalization.

05

Structured outputs

Drafts, deltas, manifests, and finals all follow a predictable artifact model.

06

Consistency at scale

The same planning logic can be applied repeatedly across requests without relying on one analyst.

EVALUATE AND EVOLVE

The skill improves through benchmarked champion-vs-challenger iteration.

STEP 1

Find gaps

Use misses, replay evidence, and fresh runs to identify planning blind spots.

→

STEP 2

Mutate skill

Adjust one bounded hypothesis: knowledge pack, rubric, template, or script.

→

STEP 3

Benchmark

Run smoke checks, benchmark cases, and comparison scoring against the champion.

→

STEP 4

Promote or reject

Accept only when quality improves without introducing blocking regression.

Decision standard **No blocking regression allowed**

Blocking cases

Critical phase contracts must continue to pass.

Defect replay

The challenger should expose known historical failure modes.

Checkpoint enforcement

Review and finalization gates must still force the right audits.

Holdout regression

Unrelated feature planning quality should not degrade.

CURRENT LIMITATIONS

The capability is already useful, but it is still optimized for a narrow operating context.

CURRENT CONSTRAINT SET

E2E-centric

The strongest patterns today are tailored to end-to-end workflow features, not every testing domain.

Prompt-sensitive

Weak instructions still lead to weak planning, even when the underlying skill is strong.

Input-dependent

Output quality tracks the quality of attached materials, runtime context, and source access.

Pack maturity matters

Coverage depth improves when feature-family knowledge packs are mature and defect-informed.

FORWARD PATH

Broaden scope without losing discipline.

- Expand beyond E2E-heavy feature families
- Improve prompt scaffolds for operators
- Keep evolving knowledge packs with real misses

→

Next stage: wider feature-family coverage with the same artifact and gate model.